

Stack and Heap in C#

Understanding Memory Management in .NET

From Fundamentals to Performance Optimization

Part I: Foundations of Memory

Chapter 1: A Brief History of Memory Management

1.1 The Early Days: Manual Memory Management

In the beginning, programmers had complete control over memory—and complete responsibility for it. Languages like C and C++ required developers to explicitly allocate memory when needed and free it when done. This was done through functions like **malloc()** and **free()** in C, or the **new** and **delete** operators in C++.

```
// C-style memory management
int* numbers = (int*)malloc(100 * sizeof(int)); // Allocate
// ... use the memory ...
free(numbers); // Must remember to free!

// C++ style
Employee* emp = new Employee(); // Allocate
// ... use the object ...
delete emp; // Must remember to delete!
```

This approach was powerful but dangerous. Forget to free memory? You have a **memory leak**—your application slowly consumes more and more RAM until it crashes or the system grinds to a halt. Free memory twice? You have a **double-free bug**—undefined behavior that could corrupt data or crash your program. Use memory after freeing it? A **use-after-free vulnerability**—a security nightmare that attackers love to exploit.

1.2 The Revolution: Garbage Collection

The idea of automatic memory management—**garbage collection**—emerged in the 1950s with John McCarthy's Lisp language. Instead of requiring programmers to track every allocation and deallocation, the runtime would automatically identify and reclaim memory that was no longer in use.

For decades, garbage collection was considered too slow for "serious" applications. Real programmers managed their own memory, or so the thinking went. But as software grew more complex and developer time became more expensive than CPU cycles, the calculus changed.

In 2002, Microsoft released the .NET Framework with the **Common Language Runtime (CLR)**—a managed execution environment with sophisticated garbage collection. C# was designed from the ground up to work with this managed memory model. Developers could focus on solving business problems instead of hunting memory bugs.

1.3 The Trade-Off

Managed memory isn't free. The garbage collector needs CPU time to run. Memory might not be reclaimed instantly. But for most applications, this trade-off is overwhelmingly positive: fewer bugs, faster development, and more reliable software.

However, understanding *how* memory management works remains crucial. Misusing memory in .NET won't cause crashes like in C++, but it can cause performance problems, memory bloat, and subtle bugs. The developers who understand the stack and heap write better, faster code.

Chapter 2: The Two Memory Regions

2.1 What Is Memory, Really?

When your .NET application runs, the operating system allocates a chunk of memory for it. This memory is a contiguous block of bytes that the application can use to store data. But not all parts of this memory are used the same way.

The CLR divides application memory into several regions, but two are particularly important for understanding how your code behaves: the **Stack** and the **Heap** (specifically, the managed heap).

2.2 The Stack: Fast, Ordered, Automatic

The stack is a region of memory that operates on a simple principle: **Last In, First Out (LIFO)**. Think of it like a stack of plates—you can only add or remove plates from the top.

Characteristic	Stack
Structure	Linear, contiguous
Access pattern	LIFO (Last In, First Out)
Allocation speed	Extremely fast (just move a pointer)
Deallocation	Automatic when scope exits
Size	Limited (typically 1MB per thread)
What it stores	Value types, references, method call info
Thread safety	Each thread has its own stack

When you call a method, a **stack frame** is pushed onto the stack. This frame contains the method's parameters, local variables (if they're value types or references), and the return address (where to continue execution after the method completes). When the method returns, its stack frame is popped off—instantly freeing all that memory.

2.3 The Heap: Flexible, Dynamic, Managed

The heap (specifically, the **managed heap** in .NET) is a larger, more flexible region of memory. Unlike the stack's strict ordering, objects on the heap can be allocated and deallocated in any order.

Characteristic	Managed Heap
Structure	Non-contiguous, dynamic
Access pattern	Random access via references
Allocation speed	Slower (requires finding space, tracking)
Deallocation	Garbage collector (non-deterministic)
Size	Limited only by available system memory

What it stores	Reference type instances (objects)
Thread safety	Shared across all threads

When you create an object with **new**, the CLR allocates space on the managed heap. The object lives there until the garbage collector determines it's no longer reachable—meaning no part of your code can access it anymore. Then, at some point, the GC reclaims that memory.

2.4 The Mental Model

Here's a simple way to think about it:

- **Stack** = Your desk. Small, fast to access, automatically cleared when you leave for the day.
- **Heap** = A warehouse. Huge, takes longer to find things, requires someone to clean up periodically.

Chapter 3: Value Types vs Reference Types

3.1 The Fundamental Distinction

Every type in C# falls into one of two categories: **value types** or **reference types**. This distinction determines where and how data is stored in memory.

Value Types

Value types directly contain their data. When you declare a value type variable, the memory for the actual data is allocated right there—typically on the stack (with some exceptions we'll discuss).

Value types include:

- Numeric types: int, long, float, double, decimal
- bool, char
- struct (including DateTime, Guid, TimeSpan)
- enum
- Tuples (ValueTuple)

```
int x = 42;           // 42 is stored directly in x's memory location
int y = x;            // y gets its own copy of 42
y = 100;              // Changing y doesn't affect x
Console.WriteLine(x); // Still 42
```

Reference Types

Reference types store a **reference** (essentially a memory address) that points to the actual data on the heap. The variable itself holds only this reference, not the data.

Reference types include:

- class
- interface
- delegate
- string (special case, immutable)
- Arrays (even arrays of value types)
- object

```
Employee emp1 = new Employee { Name = "Alice" };
Employee emp2 = emp1; // emp2 gets a copy of the REFERENCE, not the object
emp2.Name = "Bob";    // Changes the object both variables point to
Console.WriteLine(emp1.Name); // "Bob" - emp1 sees the change!
```

3.2 Memory Layout Visualized

Let's visualize what happens in memory with this code:

```
void ProcessData()
{
    int count = 5;           // Value type
    double price = 19.99;    // Value type
    Employee employee = new Employee(); // Reference type
    int[] numbers = new int[3]; // Reference type (array)
}
```

Stack:

Part II: Deep Dive into Memory Mechanics

Chapter 4: Stack Mechanics

4.1 The Stack Pointer

The stack's efficiency comes from its simplicity. The CPU maintains a **stack pointer**—a register that always points to the top of the stack. Allocation is just decrementing this pointer; deallocation is just incrementing it.

```
// Conceptual view of stack operations

void Method()
{
    int a = 1;    // Stack pointer moves down, space for 'a'
    int b = 2;    // Stack pointer moves down again, space for 'b'

    // When Method returns:
    // Stack pointer moves back up, reclaiming all space instantly
}
```

4.2 Stack Frames in Detail

Each method call creates a stack frame containing:

- **Return address:** Where to continue after the method returns
- **Parameters:** Copies of value types or copies of references
- **Local variables:** Value types and references declared in the method
- **Saved registers:** CPU state that needs to be restored

[illegible]

4.3 Stack Overflow

The stack has a fixed size (typically 1MB on Windows for .NET applications). If you exceed this—usually through deep or infinite recursion—you get a **StackOverflowException**.

```
void InfiniteRecursion()
{
    InfiniteRecursion(); // Each call adds a stack frame
    // Eventually: StackOverflowException!
}

// Also dangerous: very deep recursion
int Fibonacci(int n)
{
    if (n <= 1) return n;
    return Fibonacci(n - 1) + Fibonacci(n - 2); // Exponential stack usage
}
```

```
}
```

Important: Unlike most exceptions, `StackOverflowException` cannot be caught. The stack is corrupted; there's no safe way to continue. Your application will terminate.

4.4 Stack Allocation with `stackalloc`

C# provides **`stackalloc`** to allocate arrays on the stack instead of the heap. This can improve performance for small, short-lived arrays by avoiding heap allocation and garbage collection.

```
// Traditional heap allocation
byte[] buffer = new byte[256]; // Goes on heap, GC must clean up

// Stack allocation (C# 7.2+)
Span<byte> buffer = stackalloc byte[256]; // Goes on stack, auto-cleaned

// Use case: high-performance parsing
public int ParseInt(ReadOnlySpan<char> input)
{
    Span<char> normalized = stackalloc char[input.Length];
    // Process without any heap allocation
}
```

Tip: Use `stackalloc` for small, fixed-size buffers in performance-critical code. But be careful—too large an allocation can cause stack overflow. Generally keep `stackalloc` allocations under a few KB.

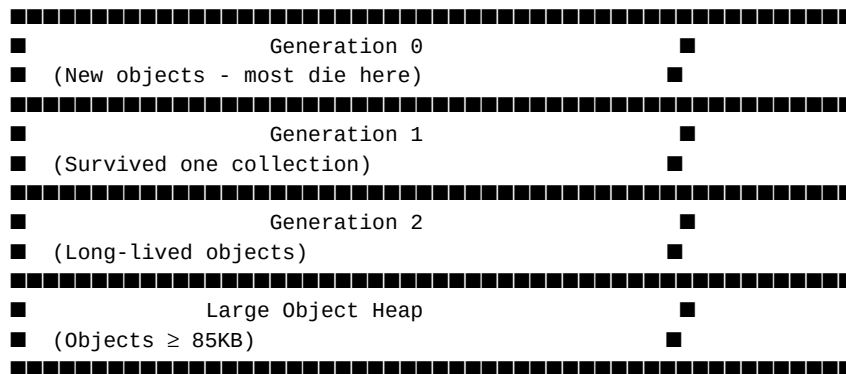
Chapter 5: Heap Mechanics and the Garbage Collector

5.1 The Managed Heap Structure

The .NET managed heap is divided into segments, primarily organized by object lifetime:

- **Generation 0 (Gen0):** Newly allocated objects. Small, frequently collected.
- **Generation 1 (Gen1):** Objects that survived one GC. Buffer between Gen0 and Gen2.
- **Generation 2 (Gen2):** Long-lived objects. Largest generation, least frequently collected.
- **Large Object Heap (LOH):** Objects $\geq 85,000$ bytes. Collected with Gen2.

Managed Heap Layout:



5.2 The Generational Hypothesis

The GC's design is based on empirical observation: **most objects die young**. In typical applications, 80-90% of objects become unreachable almost immediately after creation.

```
// Example: Most objects here are extremely short-lived
foreach (var line in File.ReadLines("data.txt"))
{
    var parts = line.Split(',');    // New string array - dies immediately
    var trimmed = parts[0].Trim();  // New string - dies immediately
    var upper = trimmed.ToUpper();  // New string - dies immediately
    ProcessValue(upper);
    // All those intermediate objects are garbage before next iteration
}
```

By focusing most collection effort on Gen0 (where most garbage is), the GC achieves efficiency. Gen0 collections are very fast—often sub-millisecond. Gen2 collections are slower but happen less often.

5.3 How Garbage Collection Works

The GC uses a **mark-and-sweep** algorithm with **compaction**:

Phase 1: Mark

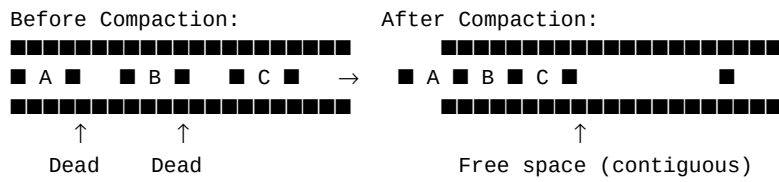
Starting from "roots" (local variables, static fields, CPU registers), the GC traces all references, marking each reachable object as "alive." Any object not marked is considered garbage.

Phase 2: Sweep

The GC reclaims memory from unmarked (dead) objects.

Phase 3: Compact

Surviving objects are moved together, eliminating fragmentation. References are updated to point to new locations.



5.4 GC Modes

Mode	Description	Best For
Workstation	Single GC thread, optimized for responsiveness	Desktop apps, UI apps
Server	One GC thread per CPU, optimized for throughput	Web servers, services
Concurrent	GC runs alongside app threads (Gen2)	Low-latency requirements
Background	Enhanced concurrent, less blocking	Default in modern .NET

Chapter 6: Boxing and Unboxing

6.1 What Is Boxing?

Boxing is the process of converting a value type to a reference type. The CLR allocates an object on the heap, copies the value into it, and returns a reference to that object.

```
int number = 42;           // Value type, on stack
object boxed = number;     // Boxing! New object on heap containing 42

// What actually happens:
// 1. CLR allocates space on heap for an object
// 2. Copies the value 42 into the heap object
// 3. Returns reference to that object
// 4. 'boxed' variable holds this reference
```

6.2 What Is Unboxing?

Unboxing is the reverse: extracting the value type from a boxed object. This requires an explicit cast and type checking.

```
object boxed = 42;
int number = (int)boxed; // Unboxing: extract the int

// What actually happens:
// 1. CLR checks that 'boxed' actually contains an int
// 2. Copies the value from heap to stack
// 3. If type doesn't match: InvalidCastException
```

6.3 The Performance Cost

Boxing and unboxing are expensive operations:

- **Memory allocation:** Every box creates a new heap object
- **Memory copy:** Data must be copied to/from the heap
- **GC pressure:** Boxed objects eventually need garbage collection

```
// Hidden boxing - ArrayList stores objects
ArrayList list = new ArrayList();
for (int i = 0; i < 1000000; i++)
{
    list.Add(i); // Boxing on every iteration!
}
// Result: 1 million heap allocations, massive GC pressure

// No boxing - List<T> stores int directly
List<int> genericList = new List<int>();
for (int i = 0; i < 1000000; i++)
{
    genericList.Add(i); // No boxing!
}
// Result: One array allocation, values stored directly
```

6.4 Common Boxing Traps

String Interpolation and Formatting

```
int count = 42;
string message = $"Count: {count}"; // Boxing in older C# versions!
```

```
// Avoid boxing with explicit ToString():
string message = $"Count: {count.ToString()}"; // No boxing

// Modern .NET 6+ often optimizes this automatically
```

Interface Calls on Structs

```
public struct Point : IFormattable
{
    public int X, Y;
    public string ToString(string format, IFormatProvider provider) => $"{{X}},{{Y}}";
}

Point p = new Point { X = 1, Y = 2 };
IFormattable f = p; // Boxing! Point is copied to heap
string s = f.ToString("", null);

// Avoid: Call method directly on struct
string s = p.ToString("", null); // No boxing
```

LINQ with Value Types

```
int[] numbers = { 1, 2, 3, 4, 5 };

// Some LINQ operations may cause boxing
var result = numbers.Cast<object>().ToList(); // Boxing every int!

// Generic LINQ avoids boxing
var result = numbers.Select(n => n * 2).ToList(); // No boxing
```

Chapter 7: Special Cases and Nuances

7.1 Strings: Reference Type, Value Semantics

Strings are reference types stored on the heap, but they behave like value types because they're **immutable**. Once created, a string cannot be changed. "Modifying" a string always creates a new one.

```
string s1 = "Hello";
string s2 = s1;           // Both reference same string object
s2 = s2 + " World";       // Creates NEW string; s1 unchanged

Console.WriteLine(s1);    // "Hello"
Console.WriteLine(s2);    // "Hello World"

// Each concatenation creates a new string
string result = "";
for (int i = 0; i < 1000; i++)
{
    result += i.ToString(); // 1000 string allocations!
}

// Use StringBuilder instead
var sb = new StringBuilder();
for (int i = 0; i < 1000; i++)
{
    sb.Append(i); // Mutates internal buffer
}
string result = sb.ToString(); // Single string allocation
```

7.2 String Interning

The CLR maintains a **string intern pool**—a table of unique string values. Literal strings are automatically interned, meaning identical literals share the same memory.

```
string a = "Hello";
string b = "Hello";
Console.WriteLine(ReferenceEquals(a, b)); // True! Same object

string c = "Hel" + "lo"; // Compiler concatenates at compile time
Console.WriteLine(ReferenceEquals(a, c)); // True! Same object

string d = "Hel";
string e = d + "lo"; // Runtime concatenation
Console.WriteLine(ReferenceEquals(a, e)); // False! Different object

// Manual interning
string f = string.Intern(e);
Console.WriteLine(ReferenceEquals(a, f)); // True! Now interned
```

7.3 Value Types on the Heap

Value types don't always live on the stack. They're stored on the heap when:

- They're fields of a reference type (class)
- They're elements of an array
- They're boxed
- They're captured by a closure (lambda/delegate)

```
public class Employee
{
```

```

        public int Id;           // int is on heap (part of Employee object)
        public DateTime Hired; // DateTime is on heap (part of Employee object)
    }

    int[] numbers = new int[100]; // The ints are on the heap (inside array)

    // Closure capture
    int counter = 0;
    Action increment = () => counter++; // 'counter' moved to heap!

```

7.4 Structs: Choosing Wisely

Use struct instead of class when:

- The type represents a single value (like int or DateTime)
- Instance size is small (Microsoft suggests ≤ 16 bytes)
- It's immutable or has no reason to be mutable
- It won't be boxed frequently

```

// Good struct: small, represents a value
public readonly struct Point
{
    public readonly int X;
    public readonly int Y;

    public Point(int x, int y) => (X, Y) = (x, y);
}

// Bad struct: too large, mutable
public struct BadExample
{
    public string Name;           // 8 bytes (reference)
    public string Address;       // 8 bytes (reference)
    public DateTime Created;     // 8 bytes
    public decimal Amount;       // 16 bytes
    // Total: 40+ bytes - too big, copying is expensive
}

```

Part III: Practical Application

Chapter 8: Memory Leaks in .NET

8.1 Wait, .NET Has Memory Leaks?

The garbage collector prevents traditional memory leaks (forgetting to free memory), but .NET applications can still have **logical memory leaks**—situations where objects are kept alive longer than necessary because something still references them.

8.2 Common Leak Patterns

Event Handler Leaks

```
public class Publisher
{
    public event EventHandler DataChanged;
}

public class Subscriber
{
    public Subscriber(Publisher pub)
    {
        pub.DataChanged += OnDataChanged; // Subscriber now referenced by Publisher
    }

    private void OnDataChanged(object sender, EventArgs e) { }

    // If Publisher lives longer than Subscriber, and we forget to unsubscribe:
    // Subscriber cannot be garbage collected!
}

// Fix: Implement IDisposable and unsubscribe
public class Subscriber : IDisposable
{
    private Publisher _publisher;

    public Subscriber(Publisher pub)
    {
        _publisher = pub;
        _publisher.DataChanged += OnDataChanged;
    }

    public void Dispose()
    {
        _publisher.DataChanged -= OnDataChanged; // Allow GC
    }
}
```

Static Collections

```
public static class Cache
{
    private static Dictionary<string, object> _cache = new();

    public static void Add(string key, object value)
    {
        _cache[key] = value; // Object now rooted forever!
    }

    // If you never remove items, they live for application lifetime
}

// Fix: Use weak references or implement expiration
```

```

public static class BetterCache
{
    private static ConditionalWeakTable<string, object> _cache = new();
    // Objects can be GC'd when no other references exist
}

```

Captured Variables in Long-Lived Delegates

```

public class Service
{
    private Timer _timer;

    public void Start()
    {
        var largeData = LoadLargeData(); // 100MB of data

        _timer = new Timer(_ =>
        {
            // Lambda captures 'largeData' - it can never be GC'd!
            Console.WriteLine(largeData.Length);
        }, null, 0, 1000);
    }
}

// Fix: Only capture what you need
public void Start()
{
    var largeData = LoadLargeData();
    var length = largeData.Length; // Capture only the length

    _timer = new Timer(_ =>
    {
        Console.WriteLine(length); // largeData can now be GC'd
    }, null, 0, 1000);
}

```

8.3 Blazor-Specific Considerations

In Blazor Server applications, each circuit (user connection) maintains state on the server. This creates unique memory management challenges:

```

// Problem: Services scoped to circuit can accumulate data
public class UserSessionService : IDisposable
{
    private List<ActivityLog> _activityLog = new();

    public void LogActivity(string action)
    {
        _activityLog.Add(new ActivityLog(action, DateTime.Now));
        // List grows unbounded during session!
    }

    public void Dispose()
    {
        _activityLog.Clear(); // Clean up on circuit disposal
    }
}

// Problem: Components that don't unsubscribe from services
@implements IDisposable

@code {
    [Inject] private INotificationService Notifications { get; set; }
}

```

```
protected override void OnInitialized()
{
    Notifications.OnNotification += HandleNotification;
}

public void Dispose()
{
    Notifications.OnNotification -= HandleNotification; // Essential!
}
}
```

Chapter 9: Performance Optimization Strategies

9.1 Reduce Allocations

The fastest allocation is the one you don't make. Every allocation is work for the GC later.

Object Pooling

```
// Without pooling: new object every request
public async Task HandleRequest()
{
    var buffer = new byte[4096]; // Allocation
    await ProcessAsync(buffer);
    // buffer becomes garbage
}

// With pooling: reuse objects
private static readonly ArrayPool<byte> Pool = ArrayPool<byte>.Shared;

public async Task HandleRequest()
{
    byte[] buffer = Pool.Rent(4096); // Get from pool
    try
    {
        await ProcessAsync(buffer);
    }
    finally
    {
        Pool.Return(buffer); // Return to pool for reuse
    }
}
```

Span<T> and Memory<T>

```
// Substring creates new string (allocation)
string input = "Hello, World!";
string hello = input.Substring(0, 5); // New string allocated

// Span creates a view (no allocation)
ReadOnlySpan<char> helloSpan = input.AsSpan(0, 5); // Just a pointer + length

// Parsing without allocations
public static int ParseFirstInt(ReadOnlySpan<char> input)
{
    int commaIndex = input.IndexOf(',');
    var firstPart = input.Slice(0, commaIndex); // No allocation
    return int.Parse(firstPart); // Span overload, no allocation
}
```

9.2 Right-Size Your Data Structures

```
// List resizing causes allocations and copies
var list = new List<int>();
for (int i = 0; i < 10000; i++)
{
    list.Add(i); // Multiple internal array resizes
}

// Pre-size when you know the count
var list = new List<int>(10000); // One allocation
for (int i = 0; i < 10000; i++)
{
    list.Add(i);
}
```

```

        list.Add(i); // No resizing needed
    }

    // Use appropriate collection types
    Dictionary<string, int> dict = new(expectedCount); // Pre-size
    HashSet<int> set = new(collection); // Initialize from existing

```

9.3 Avoid Large Object Heap When Possible

Objects $\geq 85,000$ bytes go to the LOH, which is collected less frequently and not compacted by default. This can lead to fragmentation.

```

// This array goes to LOH (85,000+ bytes)
var largeArray = new byte[100_000];

// Alternative: Use ArrayPool for temporary large arrays
var pooledArray = ArrayPool<byte>.Shared.Rent(100_000);
try
{
    // Use the array
}
finally
{
    ArrayPool<byte>.Shared.Return(pooledArray);
}

// Alternative: Stream processing instead of loading all at once
await foreach (var chunk in ReadChunksAsync(file))
{
    await ProcessChunkAsync(chunk); // Never hold entire file
}

```

9.4 Struct Optimization

```

// Pass large structs by reference to avoid copying
public readonly struct LargeStruct
{
    public readonly long A, B, C, D; // 32 bytes
}

// Copying: 32 bytes copied on each call
void ProcessCopy(LargeStruct data) { }

// Reference: Only 8 bytes (pointer) passed
void ProcessRef(in LargeStruct data) { } // 'in' = readonly reference

// Return large structs by reference
private LargeStruct _cached;
public ref readonly LargeStruct GetCached() => ref _cached;

```

Chapter 10: Debugging Memory Issues

10.1 Key Diagnostic Tools

Tool	Purpose
Visual Studio Diagnostic Tools	Real-time memory usage and snapshots
dotMemory (JetBrains)	Detailed memory profiling and leak detection
PerfView	Free Microsoft tool for GC and allocation analysis
dotnet-counters	CLI tool for real-time metrics
dotnet-dump	Capture and analyze memory dumps
dotnet-gcdump	Capture GC heap snapshots

10.2 Reading GC Metrics

```
// Monitor GC in code
public static void LogGCStats()
{
    for (int gen = 0; gen <= 2; gen++)
    {
        Console.WriteLine($"Gen{gen} collections: {GC.CollectionCount(gen)}");
    }

    var info = GC.GetGCMemoryInfo();
    Console.WriteLine($"Heap size: {info.HeapSizeBytes / 1024 / 1024} MB");
    Console.WriteLine($"Fragmentation: {info.FragmentedBytes / 1024} KB");
}

// CLI monitoring
// dotnet-counters monitor -p <pid> --counters System.Runtime
```

10.3 Investigating Memory Growth

When investigating memory issues, follow this process:

1. **Establish baseline:** What's normal memory usage?
2. **Reproduce the issue:** What actions cause growth?
3. **Take snapshots:** Before and after the action
4. **Compare:** What objects are accumulating?
5. **Trace roots:** Why can't they be collected?

```
// Force GC before taking measurements (diagnostic only!)
GC.Collect();
GC.WaitForPendingFinalizers();
GC.Collect();

long before = GC.GetTotalMemory(true);

// ... perform operation ...

long after = GC.GetTotalMemory(true);
Console.WriteLine($"Memory delta: {(after - before) / 1024} KB");
```

Part IV: Advanced Topics

Chapter 11: IDisposable and Finalizers

11.1 Unmanaged Resources

The GC manages memory, but some resources—file handles, database connections, network sockets—exist outside the managed heap. These **unmanaged resources** must be explicitly released.

```
// Without proper disposal
void ProcessFile()
{
    var reader = new StreamReader("data.txt");
    string content = reader.ReadToEnd();
    // File handle is STILL OPEN!
    // Other processes can't access the file
}

// With proper disposal
void ProcessFile()
{
    using var reader = new StreamReader("data.txt");
    string content = reader.ReadToEnd();
} // Automatically calls Dispose(), releases file handle
```

11.2 The Dispose Pattern

```
public class ResourceHolder : IDisposable
{
    private IntPtr _unmanagedHandle;
    private Stream _managedStream;
    private bool _disposed = false;

    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this); // No need for finalizer
    }

    protected virtual void Dispose(bool disposing)
    {
        if (_disposed) return;

        if (disposing)
        {
            // Free managed resources
            _managedStream?.Dispose();
        }

        // Free unmanaged resources
        if (_unmanagedHandle != IntPtr.Zero)
        {
            CloseHandle(_unmanagedHandle);
            _unmanagedHandle = IntPtr.Zero;
        }

        _disposed = true;
    }

    ~ResourceHolder() // Finalizer - safety net
    {
        Dispose(false);
    }
}
```

11.3 Finalizers: The Safety Net

Finalizers (~ClassName) are called by the GC before an object is collected. They're a safety net for when `Dispose()` wasn't called, but they have significant costs:

- Objects with finalizers survive an extra GC generation
- Finalizer thread is single-threaded (bottleneck)
- Order of finalization is undefined
- Exceptions in finalizers can crash the application

Best Practice: Always call `Dispose()` explicitly (preferably via 'using'). Only implement finalizers if your class directly holds unmanaged resources. Most classes just hold other `IDisposable` objects and don't need finalizers.

Chapter 12: Span, Memory, and Modern Memory APIs

12.1 The Span<T> Revolution

Span<T> is a stack-only type that provides a type-safe, memory-safe view into any contiguous memory region. It's the foundation of modern high-performance .NET code.

```
// Span over an array
int[] array = { 1, 2, 3, 4, 5 };
Span<int> span = array;

// Span over stack memory
Span<int> stackSpan = stackalloc int[10];

// Span over part of an array (slicing)
Span<int> middle = array.AsSpan(1, 3); // { 2, 3, 4 }

// All operations are bounds-checked and safe
span[0] = 100; // Modifies original array
// span[10] = 0; // Throws IndexOutOfRangeException
```

12.2 ReadOnlySpan<T>

ReadOnlySpan<T> is the immutable version, commonly used for strings:

```
// Zero-allocation string operations
string text = "Hello, World!";
ReadOnlySpan<char> span = text.AsSpan();
ReadOnlySpan<char> hello = span.Slice(0, 5); // No new string!
ReadOnlySpan<char> world = span.Slice(7, 5); // No new string!

// Efficient parsing
public static bool TryParseCoordinate(ReadOnlySpan<char> input,
    out double lat, out double lon)
{
    int comma = input.IndexOf(',');
    if (comma < 0) { lat = lon = 0; return false; }

    return double.TryParse(input.Slice(0, comma), out lat) &&
        double.TryParse(input.Slice(comma + 1), out lon);
}
```

12.3 Memory<T> for Async Operations

Span<T> is stack-only—it can't be used in async methods or stored in fields. Memory<T> is the heap-friendly equivalent.

```
// Span can't be used in async methods
async Task ProcessAsync(Span<byte> data) // Compiler error!
{
    await Task.Delay(1);
}

// Memory<T> works with async
async Task ProcessAsync(Memory<byte> data)
{
    await Task.Delay(1);

    // Get a Span when you need to work with the data
    Span<byte> span = data.Span;
    ProcessSync(span);
}
```

```

}

// Common pattern: IMemoryOwner<T> for pooled buffers
using IMemoryOwner<byte> owner = MemoryPool<byte>.Shared.Rent(1024);
Memory<byte> memory = owner.Memory;
await ProcessAsync(memory);

```

12.4 Practical Example: Zero-Copy CSV Parsing

```

public static IEnumerable<(ReadOnlyMemory<char> Name, int Value)>
    ParseCsv(ReadOnlyMemory<char> csv)
{
    var remaining = csv;

    while (!remaining.IsEmpty)
    {
        int newline = remaining.Span.IndexOf('\n');
        var line = newline < 0 ? remaining : remaining.Slice(0, newline);

        int comma = line.Span.IndexOf(',');
        if (comma > 0)
        {
            var name = line.Slice(0, comma);
            if (int.TryParse(line.Slice(comma + 1).Span, out int value))
            {
                yield return (name, value);
            }
        }

        remaining = newline < 0
            ? ReadOnlyMemory<char>.Empty
            : remaining.Slice(newline + 1);
    }
}

// Usage: Zero heap allocations during parsing!
var csv = File.ReadAllText("data.csv").AsMemory();
foreach (var (name, value) in ParseCsv(csv))
{
    // name is a view into the original string
}

```

Chapter 13: Records, ref structs, and Modern C#

13.1 Records: Immutable Reference Types

Records (C# 9+) are reference types designed for immutable data. They provide value-based equality while remaining on the heap.

```
public record Person(string Name, int Age);

// Record behavior:
var p1 = new Person("Alice", 30);
var p2 = new Person("Alice", 30);
Console.WriteLine(p1 == p2); // True (value equality, unlike classes)

// Non-destructive mutation (creates new instance)
var p3 = p1 with { Age = 31 }; // New object on heap

// Still a reference type
Person p4 = p1;
Console.WriteLine(ReferenceEquals(p1, p4)); // True (same object)
```

13.2 Record Structs: Value Type Records

```
public readonly record struct Point(int X, int Y);

// Combines struct benefits with record syntax:
// - Stack allocation (when local)
// - Value equality
// - Readonly (immutable)
// - Deconstruction

var p1 = new Point(1, 2);
var p2 = new Point(1, 2);
Console.WriteLine(p1 == p2); // True

var (x, y) = p1; // Deconstruction
```

13.3 ref struct: Stack-Only Types

ref structs are guaranteed to live only on the stack. This makes them suitable for high-performance scenarios where heap allocation must be avoided.

```
public ref struct StackOnlyBuffer
{
    private Span<byte> _buffer; // Can hold Span<T>!

    public StackOnlyBuffer(Span<byte> buffer)
    {
        _buffer = buffer;
    }
}

// Restrictions on ref structs:
// - Cannot be boxed (no object, no interfaces)
// - Cannot be fields in classes or non-ref structs
// - Cannot be used in async methods
// - Cannot be captured by lambdas
// - Cannot be elements of arrays

// Span<T> itself is a ref struct!
Span<int> span = stackalloc int[10]; // Must stay on stack
```

Appendix A: Quick Reference

Stack vs Heap Summary

Aspect	Stack	Heap
Access Speed	Very fast	Slower
Allocation	Automatic (push)	GC managed (new)
Deallocation	Automatic (pop)	Garbage collector
Size Limit	~1MB per thread	System memory
What Lives Here	Value types*, references	Objects, arrays
Thread Safety	Thread-private	Shared
Fragmentation	None (LIFO)	Possible

**When declared as local variables or parameters, not as class fields*

Type Categories

Category	Examples	Default Location
Value Types	int, double, bool, struct, enum	Stack (if local)
Reference Types	class, interface, delegate, string, array	Heap (always)
ref struct	Span<T>, ReadOnlySpan<T>	Stack (enforced)

GC Generation Triggers

Generation	Triggers	Typical Frequency
Gen0	When Gen0 budget exceeded	Very frequent (ms)
Gen1	When Gen1 budget exceeded	Less frequent
Gen2	When Gen2 budget exceeded, LOH allocation	Infrequent (seconds)
Full GC	Memory pressure, GC.Collect()	Rare

Appendix B: Further Reading

1. **"Pro .NET Memory Management"** by Konrad Kokosa — The definitive deep dive into .NET memory
2. **"Writing High-Performance .NET Code"** by Ben Watson — Practical optimization techniques
3. **"CLR via C#" by Jeffrey Richter** — Foundational understanding of the runtime
4. **Microsoft Docs: Memory Management** — <https://docs.microsoft.com/en-us/dotnet/standard/garbage-collection/>
5. **Adam Sitnik's Blog** — In-depth performance analysis posts
6. **.NET Blog: Performance** — Updates on runtime improvements

This guide was written to help .NET developers understand the memory model that underpins their applications. While the garbage collector handles most complexity, understanding these concepts leads to better architectural decisions, more efficient code, and easier debugging of memory-related issues.